

Experimental Analysis of TimSort as a Hybrid Sorting Algorithm

Biloliddin Ubaydulloev

Kaskelen, 2026

I. Abstract

Sorting is really important in computer science because it helps with data processing and making searches better. This study looks at how sorting algorithms work, especially TimSort which is a mix of different approaches. The study compares how QuickSort MergeSort, HeapSort and TimSort work and also looks at some new ideas from artificial intelligence that can help make sorting better. The study tried out these algorithms on types of data like random data, sorted data and data that is partly sorted to see how they work in different situations. It looked at things like how it takes to run how much memory it uses, if it is stable and if it can handle large amounts of data. The results show that TimSort is really good at sorting data that is partly sorted or data from the world because it can change how it works based on the data. It is also very efficient when dealing with amounts of data. The study also found that using ideas from intelligence can make sorting even better by choosing the best way to sort data at the time. This study helps us understand the bad things about each algorithm and shows why it is important to use a mix of different approaches for applications that use a lot of data. It gives us advice on how to choose the right sorting method based on the type of data we are working with and what our computer can handle. The results of this study are important because they help us see that TimSort is a choice for many types of data especially data that is partly sorted. This is because TimSort can change how it works to fit the data, which makes it very efficient. The study also shows that using intelligence can help make sorting better which is a big deal, for applications that use a lot of data. Overall this study helps us understand how to choose the sorting algorithm for our needs.

II. Objectives

The primary goals of this research are:

- To examine the operational concepts of conventional sorting algorithms like QuickSort, MergeSort, HeapSort, and TimSort [1].

- To assess the efficiency of these algorithms regarding time complexity, memory utilization, and scalability [2].
- To explore how AI methods can improve sorting algorithms [3].
- To evaluate conventional and AI-driven sorting methods across various data scenarios [2] [3].
- To determine the benefits, drawbacks, and appropriate applications for every algorithm [1] [2].
- To offer insights on optimizing sorting algorithms for contemporary applications that utilize extensive datasets [3].

III. Introduction

In today's data-driven era, the rapid growth of data generation has created significant challenges in processing large volumes of information efficiently. Modern industries rely heavily on fast data processing for applications such as search, analytics, and decision-making. Sorting operations represent a fundamental component of these processes, as their efficiency directly affects the performance of subsequent computations. However, traditional sorting algorithms often struggle to maintain high efficiency when dealing with massive and complex datasets, leading to performance degradation in large-scale systems.

Sorting algorithms such as QuickSort and MergeSort have been widely studied and applied due to their strong theoretical foundations and practical performance. Nevertheless, as data characteristics become more diverse and unpredictable, selecting the most suitable algorithm becomes increasingly difficult. The effectiveness of a sorting algorithm depends on multiple factors, including dataset size, distribution, memory constraints, and stability requirements.

Recent advancements have introduced hybrid and adaptive approaches to address these limitations. TimSort, for example, combines MergeSort and Insertion Sort to exploit existing order in data, making it highly efficient for real-world datasets. In addition, emerging research explores artificial intelligence (AI)-based enhancements that dynamically adapt algorithm behavior, improving performance in complex and large-scale environments.

This study aims to conduct a comparative and experimental analysis of several sorting algorithms, including QuickSort, MergeSort, HeapSort, and TimSort. The research evaluates their performance across datasets of varying sizes and distributions, including random, sorted, and partially sorted data. Key performance metrics such as execution time, memory usage, algorithmic complexity, and stability are analyzed to determine the strengths and limitations of each approach.

Furthermore, this research investigates optimization techniques such as data partitioning and parallel processing to improve scalability in big data environments. By analyzing both traditional and adaptive strategies, this study provides practical insights into selecting the

most appropriate sorting algorithm based on dataset characteristics and computational requirements.

The remainder of this paper is organized as follows: [2] reviews related work and existing research on sorting algorithms and their optimizations. [3] describes the methodology and implementation details. [3] presents the experimental results and analysis. Finally, Section 5 concludes the study and discusses future research directions.

IV. Problem Statement

Conventional sorting algorithms, while effective in various situations, frequently show performance constraints when handling diverse and extensive datasets. Their effectiveness significantly relies on factors like data distribution, size, and sequence. For instance, algorithms such as QuickSort can perform poorly in worst-case situations because of ineffective pivot choices [2].

Moreover, traditional algorithms are not adaptable—they do not modify their strategies dynamically according to the characteristics of the input data. This constraint becomes crucial in practical applications where data tends to be unpredictable and constantly evolving [1].

Thus, the challenge is to assess if AI-optimized sorting algorithms can surpass these constraints by implementing adaptive, data-informed decision-making processes. This research intends to assess if these methods can reliably exceed conventional algorithms regarding efficiency, scalability, and resilience [3].

V. Related Work

Sorting algorithms are very important in computer science. Many people have studied them. QuickSort, MergeSort and HeapSort are well-known. They help process data. QuickSort, made by Tony Hoare works on average. MergeSort, made by John von Neumann always works well. HeapSort is good. Not as fast as the others. Some new sorting methods combine ones. For example TimSort, made by Tim Peters mixes MergeSort and Insertion Sort. It is good for data that is almost sorted. That is why it is used in programming languages. Recently people have tried to make sorting better using machine learning and artificial intelligence.

These technologies help choose the sorting method. They can also change how the sorting works while it is happening. This makes sorting for big and complicated data. Some researchers are trying things. They think of sorting as a problem that needs a decision. They use a type of machine learning called deep reinforcement learning. This helps them find ways to sort data. Others are working on uses. For example they are making help desk systems and task scheduling. Some people are working on sorting data in a way. They use math and encryption. This way data can be sorted without being seen by others. Sorting data on computers at once is also being studied. New methods can spread the work across computers. This makes sorting some new ideas can even sort data in a way. Overall people have made sorting algorithms better. There is still more to do. People need to make sorting work in the real world. It needs to be more adaptable and scalable. Effect of Pre-Sorted Data on TimSort :

% Pre-Sorted Data	TimSort Runtime (ms)
0% (Random)	1150
25%	1000
50%	850
75%	700

VII . METHODOLOGY

A. Dataset Description and Characteristics

This study used two kinds of datasets to see how well sorting algorithms work. It focused mainly on TimSort. The datasets were made to show kinds of input like how big they are, how the numbers are spread out and how much order they have. They made datasets using four different kinds of distributions: uniform, normal, exponential and bimodal. These synthetic datasets came in all sorts of sizes from 1,000 elements, to 1,000,000 elements to see how well the algorithms could handle bigger loads. They also used real-world datasets to make sure the study was practical and relevant. These real-world datasets consisted of:

- Iris dataset (150 records)
- Wine Quality dataset (~6,500 records)

- Student dataset (~5,800 records)
- MNIST subset (~42,000 records)
- Yelp Reviews dataset (~10,000 records)

These datasets represent structured, semi-structured, and high-dimensional data, allowing a comprehensive evaluation of sorting behavior in realistic scenarios.

B. Experimental Setup and Implementation

The experiment was implemented in Python, where four sorting algorithms were selected for comparison:

- QuickSort
- MergeSort
- HeapSort
- TimSort

I used the logic for all the algorithms so they can be compared fairly. The TimSort that comes with Python, which is used when you call the function, was used as the standard version of TimSort to compare with. I ran each algorithm on every set of data times ten times, for each test and I wrote down the average time it took to run. This helps to make sure the results are not random and are reliable. I used Python's time module to measure the time it took to run each algorithm because it is very precise and gives results.

Experimental Environment

All experiments were conducted under a controlled environment with the following specifications:

- Processor: Intel Core i7 (10th Gen)
- RAM: 16 GB
- Operating System: Windows 11
- Programming Language: Python 3.10

All tests were executed in a single-threaded environment to ensure consistent and comparable performance results.

C. Evaluation Metrics

The performance of the algorithms was evaluated using the following metrics:

- **Execution Time:** Time required to sort each dataset.
- **Scalability:** Performance variation as dataset size increases.
- **Stability:** Ability to preserve relative order of equal elements.
- **Memory Efficiency:** Qualitative assessment of additional memory usage.

These metrics were selected to provide a balanced evaluation of both theoretical and practical performance.

D. Limitations of the Methodology

This experiment has some limitations. First we did this experiment in Python. So the results might be different in programming languages or if we made some changes to make it run faster. We only used a datasets to test this. These datasets might not be like the data we see in life. We looked at a different algorithms to see how they work.. We were really looking at TimSort. So we made sure our test was good for comparing TimSort to algorithms. We were not trying to test all algorithms. We mostly looked at how it took to run the experiment. We did not look deeply at things like how much work the computers brain was doing or how the system was working. With these limitations our method is a good way to compare TimSort to other sorting algorithms. The TimSort algorithm is what we were focused on. We wanted to make sure our test was fair and consistent, for evaluating TimSort.

VIII. RESULTS

A. Runtime Comparison

The performance of sorting algorithms was looked at to see how long they take to run on sized datasets. This study is mainly about how TimSort does compared to algorithms like QuickSort, MergeSort and HeapSort. The results of the experiments show that TimSort always takes time to run on datasets that are almost sorted and look like real world data. This is because TimSort can adapt to the data it is sorting. MergeSort does a job on all types of datasets. QuickSort is fast on datasets but gets

much slower when the data is not sorted well like when it is sorted in reverse. When the datasets are very big like one million elements TimSort does the best. MergeSort is behind. HeapSort is always slower because it has to rebuild the heap times. TimSort does well on datasets and on datasets that are almost sorted which makes it a good choice. The performance of TimSort and other algorithms, like QuickSort and MergeSort can vary depending on the dataset.

B. Stability and Memory Analysis

The stability and memory usage characteristics of each algorithm were also analyzed.

Algorithm	Stability	Memory Usage	In-place
QuickSort	Not stable	Low	Yes
MergeSort	Stable	High	No
HeapSort	Not stable	Low	Yes
TimSort	Stable	Medium	Yes

TimSort maintains stability while keeping memory usage moderate, making it more suitable for real-world applications where preserving data order is important.

C. Performance Analysis Summary

The results indicate clear differences in algorithm behavior:

- **QuickSort** performs well on small and random datasets but degrades in worst-case scenarios.
- **MergeSort** provides consistent performance regardless of dataset distribution.
- **HeapSort** is memory efficient but slower in practice due to re-heapification overhead.
- **TimSort** shows the most balanced and adaptive performance, especially on partially sorted datasets.

TimSort is really good at handling world data because it is a mix of Merge Sort and Insertion Sort. This makes TimSort very useful, for things where the data is already partly sorted. TimSort works with this kind of data because it is a combination of Merge Sort and Insertion Sort.

D. Key Findings

TimSort works well with data that is already partly sorted like in real-world situations. MergeSort gives reliable results in all cases. QuickSorts performance can vary a lot depending on the data it is sorting. HeapSort does not work well in real life even though it seems good on paper. These findings confirm that hybrid algorithms such as TimSort are more suitable for modern data-intensive environments compared to purely traditional sorting approaches.

IX. Conclusion

This study looked at how sorting algorithms work and it focused on TimSort. The main goal was to see how TimSort works compared to algorithms like QuickSort, MergeSort and HeapSort when used with different types of data. The study found that TimSort always does a job and works well especially when the data is already partly sorted or comes from real life. This is because TimSort uses two methods: one that merges things and another that optimizes by inserting. This helps TimSort work with the data it is given. For example QuickSort is great with amounts of data or data that is all mixed up but it does not work as well when the data is already sorted. MergeSort is very stable. You can count on it but it needs more memory to work which can be a problem sometimes. HeapSort uses memory but it is slower than the other methods. The study also found that new ideas or methods inspired by intelligence can sometimes make things work better but they can also make things more complicated and use more computer power. This means they might not be practical for use. Overall the study shows that methods like TimSort that combine approaches are a good choice because they are fast, stable and work well. This makes TimSort very useful, for real-life applications where there is a lot of data that's not always random. In the future it would be interesting to see how sorting algorithms can be improved using methods that adapt to the data or machine learning especially when working with data systems or data that is spread across many computers.

References:

- [1] Sabah, A. S., Abu-Naser, S. S., Helles, Y. E., Abdallatif, R. F., Samra, F. Y. A., Taha, A.H.A., Massa, N. M., & Hamouda, A. A. (2023).
- [2] Al-Aydi, B. M., & Abu-Naser, S. S. (n.d.). Comparative study of traditional and AI-enhanced sorting algorithms: QuickSort, MergeSort, HeapSort, and TimSort.
- [3] Li, Y. (2025). Performance analysis of efficient sorting algorithms in big data processing. *Procedia Computer Science*, 262, 44–50.